

# Hadoop Framework

Irene Siragusa, PhD

# Outline

---

-  Hadoop
-  HDFS
-  YARN
-  MapReduce Algorithm

# Hadoop

---

- Apache project, implements an open-source trustworthily, scalable and distributed framework for data storage and elaboration.
- Apache Hadoop it is a distributed framework for Big Data
- It allows distributed elaboration over big data sets over clusters via easy models of programming.
- <https://hadoop.apache.org/docs/stable/>

# Hadoop

---

- Designed for scaling from single server to thousands of machines, each of them offer local computational and storage resources.
  - High availability is offered from the framework and not rely on hardware.
  - Framework is designed to detect and manage failures at application level, thus offering an high availability service over a cluster of computers, where each of them can be prone to failures.
- Data software architecture for batch-oriented elaborations.
  - Distributed File system – [HDFS](#)
  - Resource manager – [YARN](#)
  - Batch processor – [MapReduce](#)

# Hadoop

---

- Interface with a series of other software components, also developed by Apache, which extend its functionality to different fields of application:
  - Stream processing
  - Workflow management
  - Interacting with RDBMS
  - Machine learning
- Hadoop ecosystem, can be considered as the sum of all these components

# Hadoop – features

---

- Ability to store and process fast a huge volume of data of any type
  - A fundamental characteristic to manage the constant increase in volumes and variety of data, in particular social media and IoT.
- Powerful computation
  - Thanks to Hadoop's distributed computational models for fast processing of big data. The more computing nodes you use, the more processing power you will have.
- Fault tolerance
  - Data and its processing are protected from hardware failures. If one node fails, tasks are automatically routed to other nodes to ensure that distributed computing works.
  - Each data is automatically saved in multiple copies.

# Hadoop – features

---

- Flexibility

- There is no need to process data before storing it. Data can be stored directly and how to use them can be decided later.
- This includes unstructured data, such as text, images, and videos.

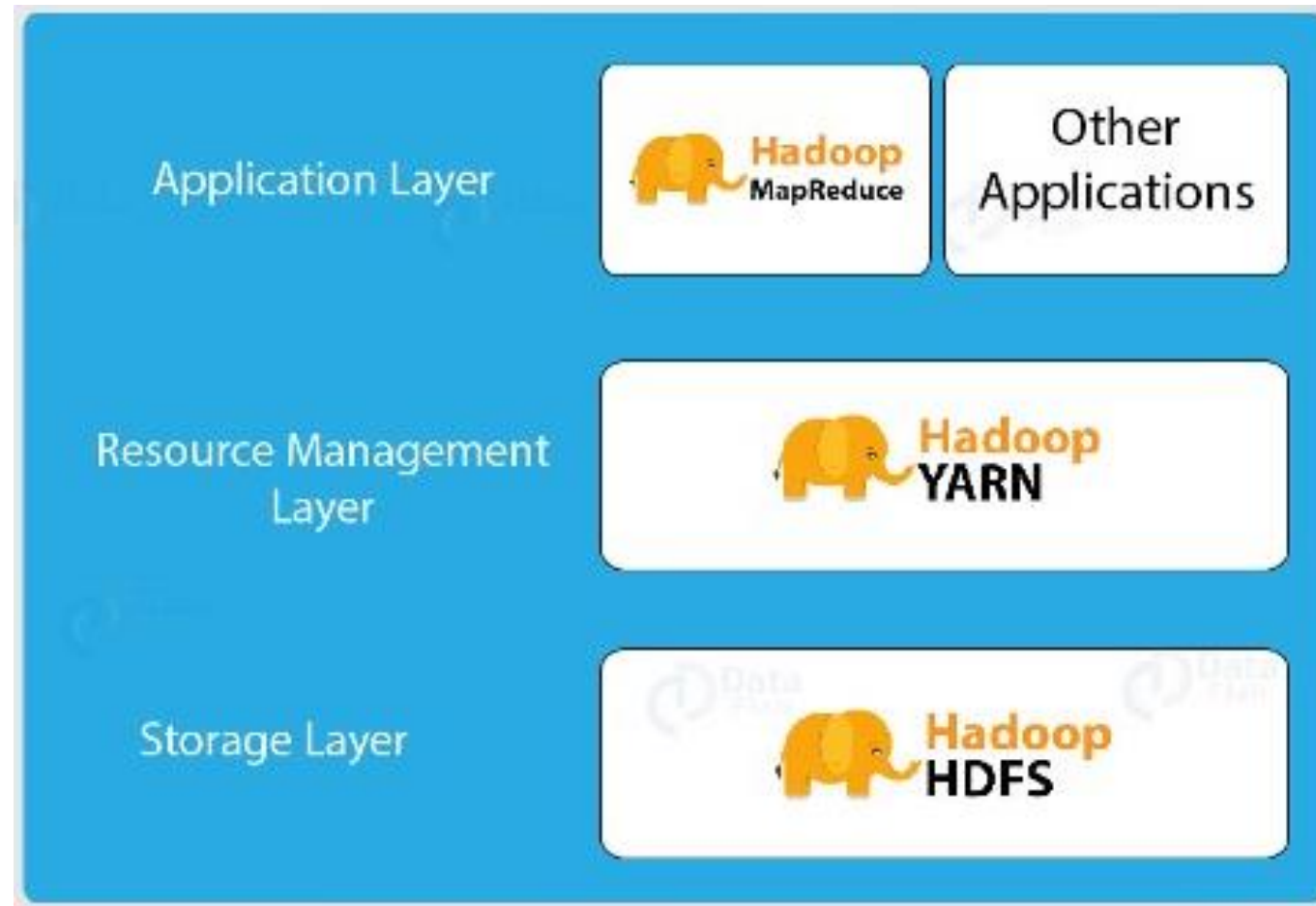
- Low cost

- The open-source framework is free and uses commodity hardware to store large volumes of data.

- Scalability

- You can easily augment the data managed by your system just by adding new nodes. The administration required is very minimal.

# Hadoop – architecture



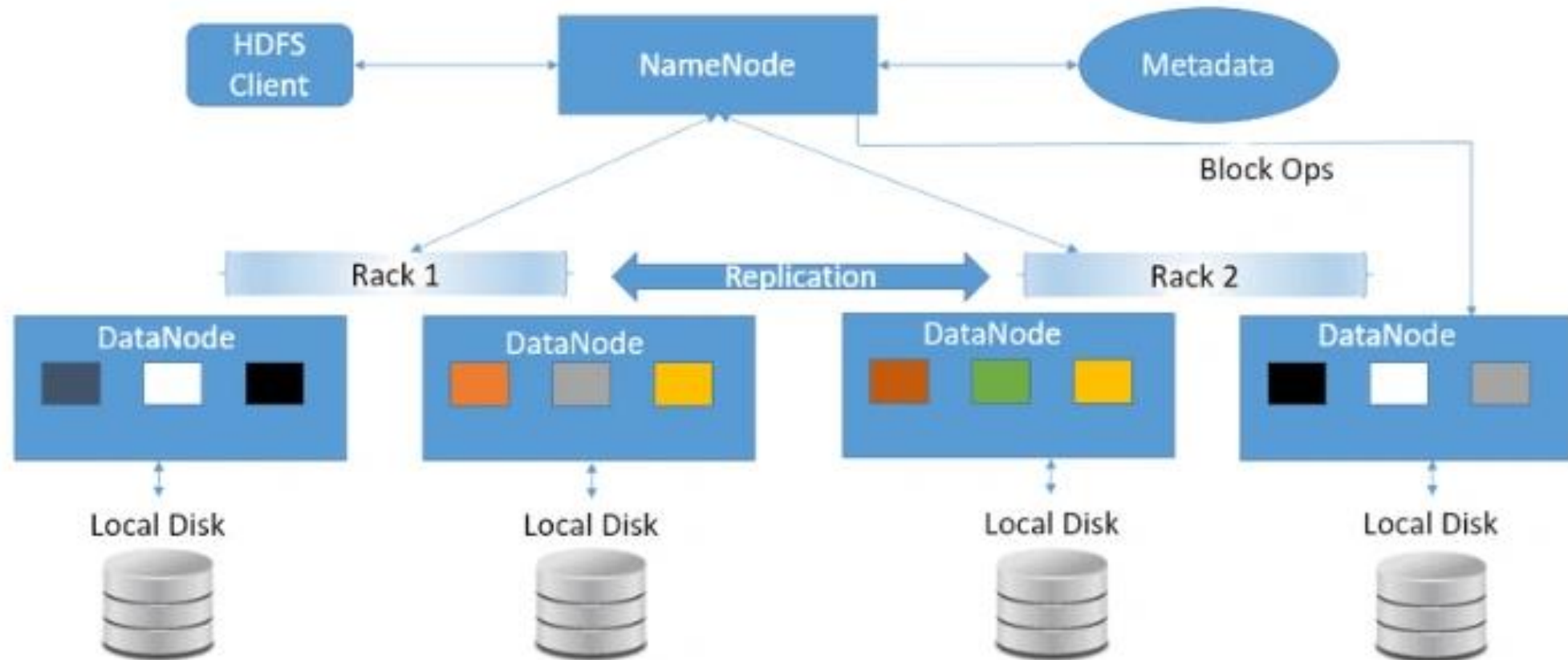


# Hadoop – architecture

---

- Distributed, with some centralization
- Core nodes of the cluster are where most of the system's computing power and storage resides
- Core nodes run TaskTracker to accept and respond to MapReduce tasks and also DataNode to store the necessary blocks as close as possible
- The central control node runs NameNode to track directories and HDFS files, and JobTracker to send computational tasks to TaskTracker
- Written in Java, it also supports Python

# Hadoop – architecture



# HDFS

---

- Hadoop Distributed File System is a portable, scalable and distributed file system, written in Java for Hadoop framework.
- A cluster in Hadoop typically has one or more
  - NameNodes, on which the file metadata resides
  - a set of DataNodes, on which the HDFS files reside, in blocks of fixed size.

# HDFS

---

- HDFS philosophy is to keep a few large fragments of a file in the infrastructure, rather than many small fragments.
- Most commonly used file formats on HDFS are Comma-Separated Values (CSV) but also Apache Avro, Apache ORC, and Apache Parquet.

# HDFS – characteristics

---

- Fault tolerance
  - Typically, an HDFS instance runs on thousands of nodes, and there are internal mechanisms for anomaly detection
- High throughput processing
  - Designed for batch processing, it compensates for low latency with high throughput for streaming processing

# HDFS – characteristics

---

- Large amounts of data
  - The scalability of the nodes, the large block size and the high put throughput allow the management of data sets in the order of terabytes
- Write-once-read-many
  - A file tends to be created and written and then not modified
  - Adapts to the MapReduce model: the mapper writes and the reducer reads

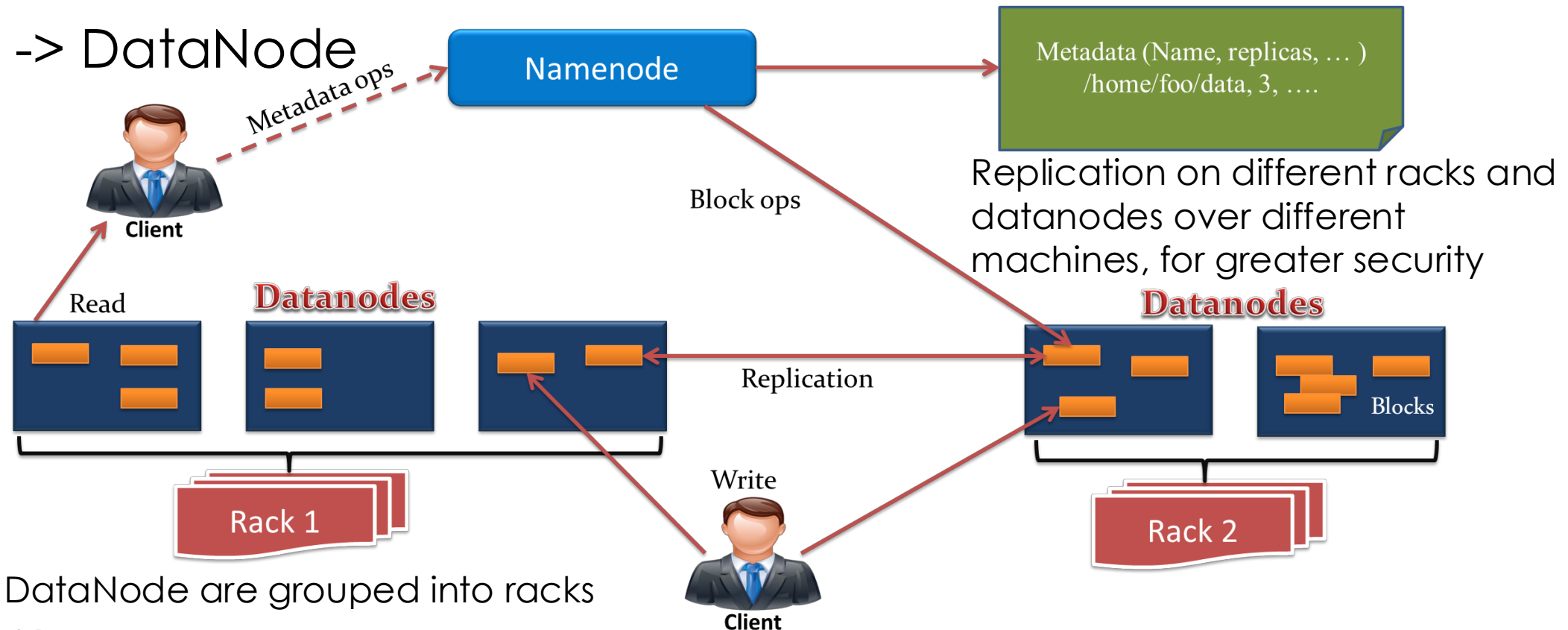
# HDFS – characteristics

---

- Computation close to data
  - It is easier to move the computation close to the data and not vice versa
  - The bandwidth used is reduced
- Portability
  - Written specifically for this purpose
  - Java-based
- Tailored on MapReduce's needs
- Replication factor for each block is 3 by default
- Each file is divided into blocks (e.g. 128 MB)

# HDFS – architecture

- Master -> NameNode
- Slave -> DataNode





# HDFS – architecture

---

- NameNode
- Secondary NameNode
- DataNode
- Checkpoint Node
- Backup Node

# HDFS – architecture: NameNode

---

- It is the *manager* of the DataNodes and is their *abstraction*
- Transaction log for deleting/adding files, etc.
- It stores metadata for files, such as the directory structure.

# HDFS – architecture: NameNode

---

- It stores metadata for files, such as the directory structure.
  - FsImage
    - It stores the entire filesystem namespace in a sort of "image file" in the NameNode's local file system.
    - It contains a serialized representation of all the folders and nodes where the metadata actually resides
    - Static file
  - EditLogs
    - It contains all recent file system changes to the latest version of FsImage
    - This file records all create/update/delete requests from DataNodes
    - Dynamic file

# HDFS – architecture: NameNode

---

- It regulates client access to files
- It executes file system's operations: opening, closing, renaming of files and folders
- It manages signals from DataNodes:
  - Heartbeat coming from an active DataNode
  - Block report that contains a list with all the active DataNodes
- It defines the replication factor for each blocks
- It manages the creation of replication blocks if needed after a DataNode error

# HDFS – architecture: Secondary NameNode

---

- It is used for speed up restart time of NameNode, in case of failure
- It is recalled when NameNode is down
- At startup time, HDFS state is read from FsImage and then changes from EditLogs are applied
  - NameNode makes a merge from both files and can be long due to their dimensions
  - Secondary NameNode performs an independent merge and saves everything as in persistent storage
- It is regularly synchronized with NameNode

# HDFS – architecture: DataNode

---

- It is the slave of the architecture
- It saves the data in HDFS
- It can be executed over any filesystem (ext4, NTFS)
- It notifies to NameNode the blocks it manages
- Every 3 sec it sends an heartbeat signal to NameNode
- It executes read/write operations, creates blocks repetitions and manages their deletion

# HDFS – architecture: Checkpoint Node

---

- It creates periodical checkpoints of the namespace
  - It downloads FsImage and EditLogs files from NameNode and performs the local merge
  - It updates NameNode via uploading the new FsImage file
  - It stores the checkpoint in a folder structure equal to the NameNode ones to make it always immediately available to the NameNode itself if it is necessary

# HDFS – architecture: Backup Node

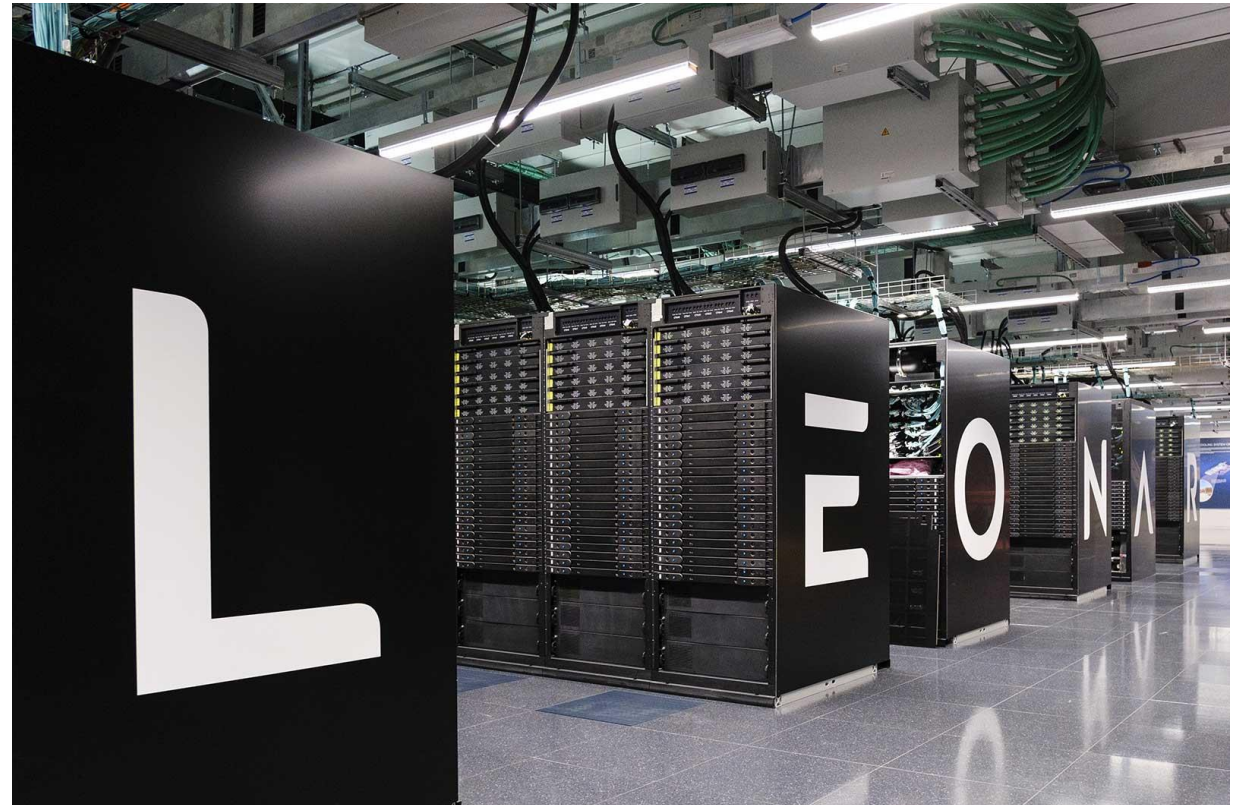
---

- It keeps an updated copy of the namespace in central memory
- It is always synchronized with NameNode
- No explicit checkpoint are needed: it downloads FsImage and EditLogs files and locally do their merge
- There is just one backup node at time for each Namenode



# HDFS – architecture: Rack

- It is a cluster of physical machines over which a set of DataNodes was deployed
- First replication is done on the local node that created it, the second one in another DataNode on the same rack and the third over a DataNode in another rack

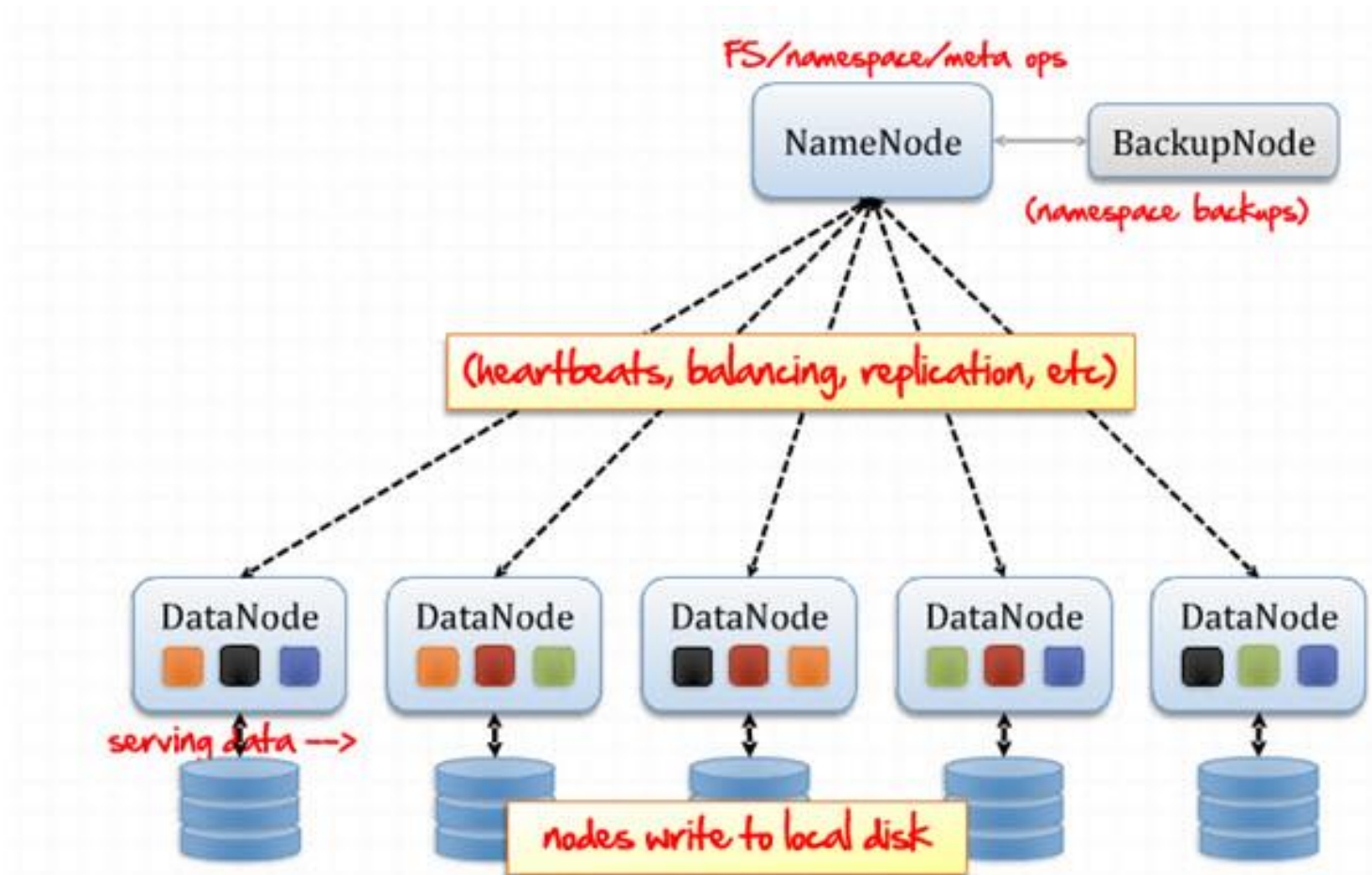


# HDFS – architecture: Rack

---

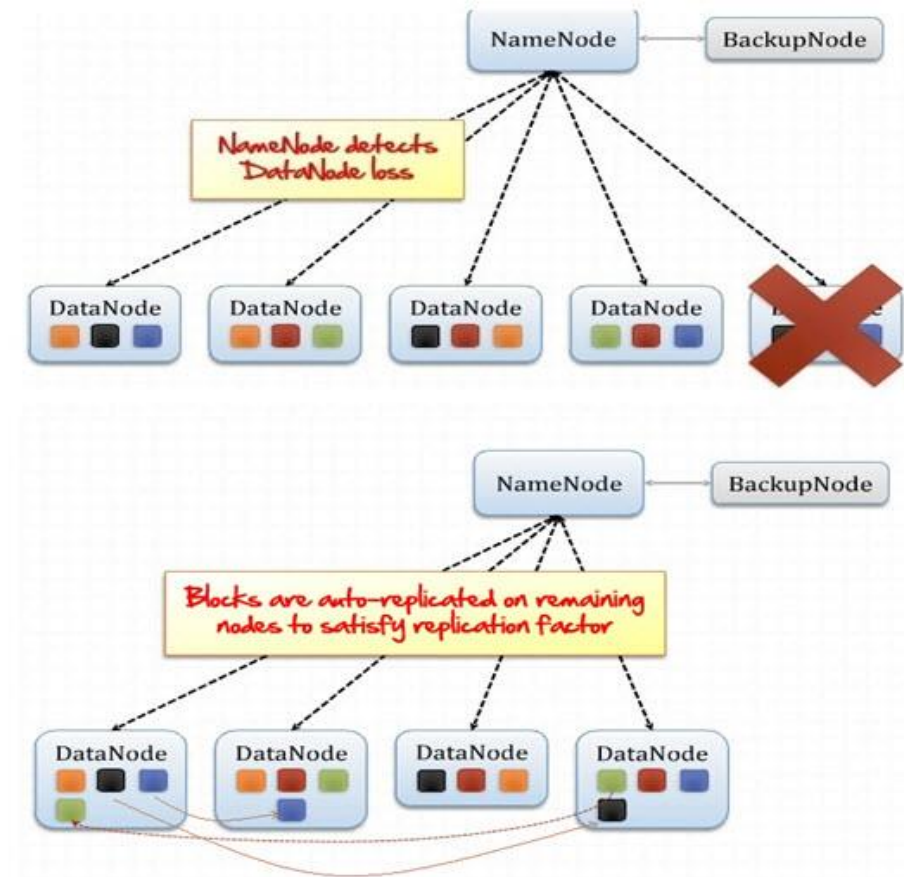
- Rack awareness is an algorithm offering a criteria of chose of the closest DataNode to the NameNode to create a new block
  - It is based on the knowledge of the rack-id of each DataNode
  - It limitates the occupied bandwidth
  - It reduces the costs of read-write operations

# HDFS – architecture



# HDFS – architecture: Data Availability

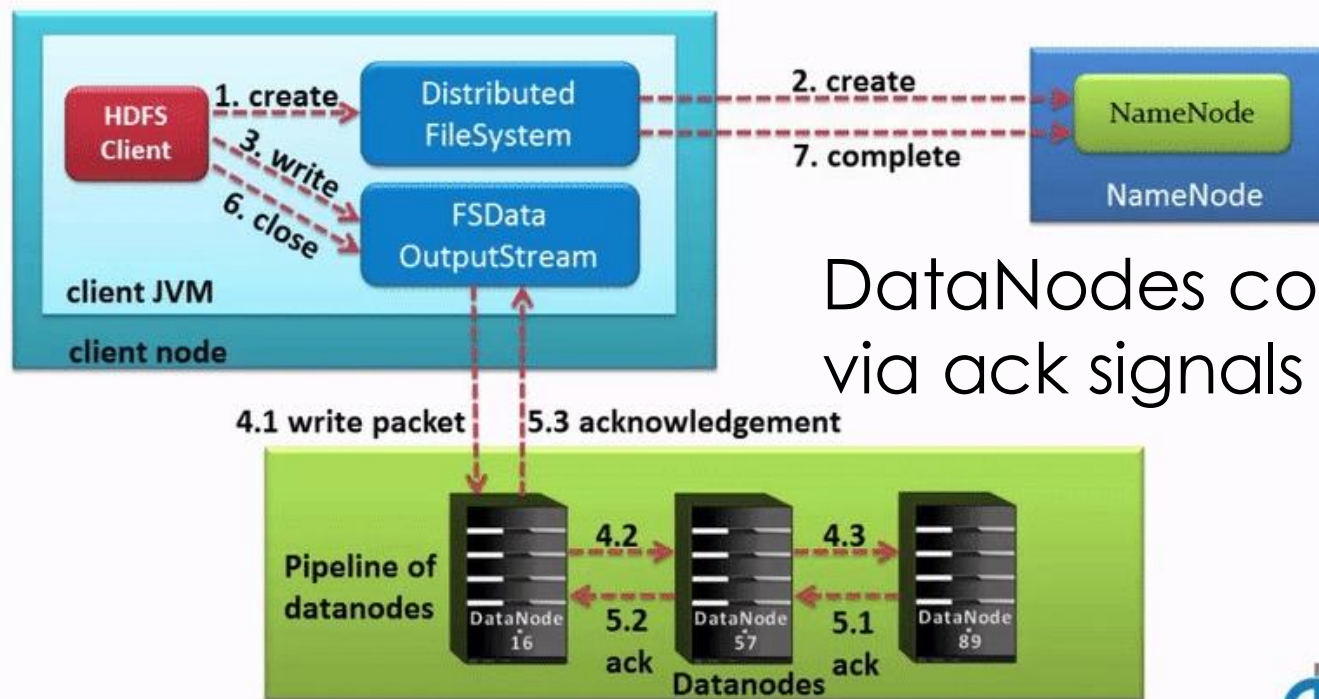
- Client requires a block from a DataNode, if no answer is obtained, a failure on that specific DataNode is notified, and needed blocks are required from their replications over other DataNodes, thus assuring availability



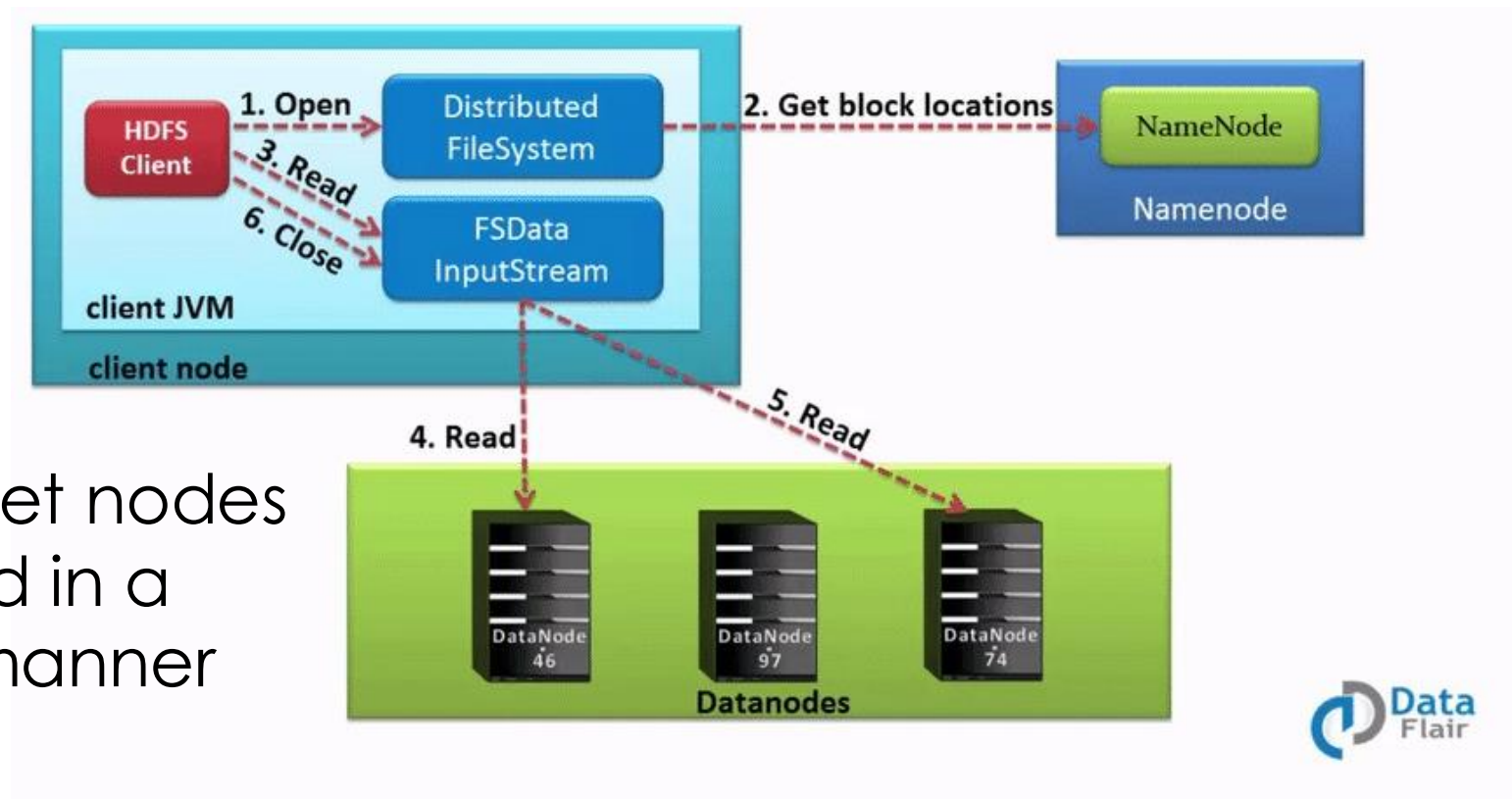


# HDFS – write operations

1. For the first time a NameNode is created
2. Creates NameNode Instance



# HDFS – read operations



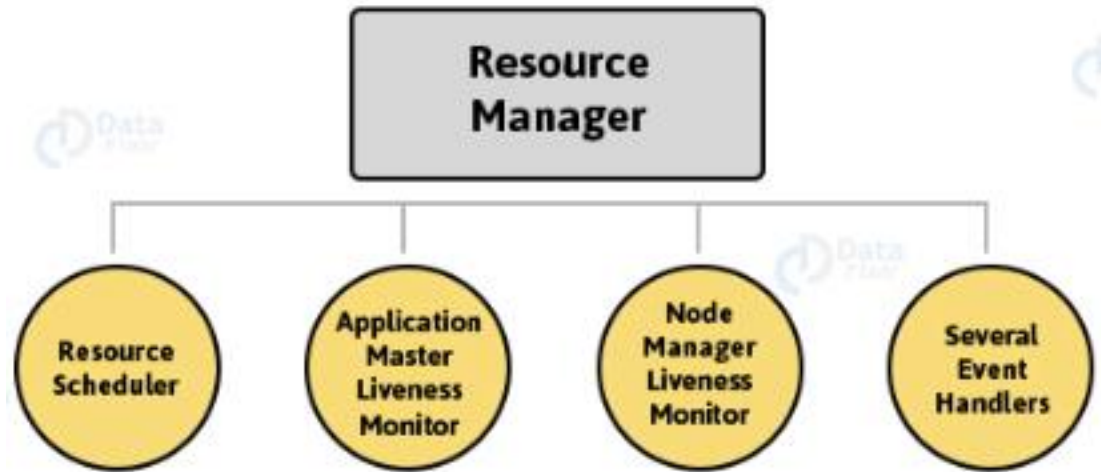
# YARN

---

- Yet Another Resource Negotiator (YARN)
- It has three principal components
  - Resource manager of clusters resources
  - Node manager for single's node resources
  - Job submitter

# YARN – Resource manager

- It manages rack awareness
- It is aware of the real resources for each slave and manage them (Resource Scheduler)
- Application Manager service acquires the first container for each application via Node manager
- Application manager manages the execution of a target MapReduce task

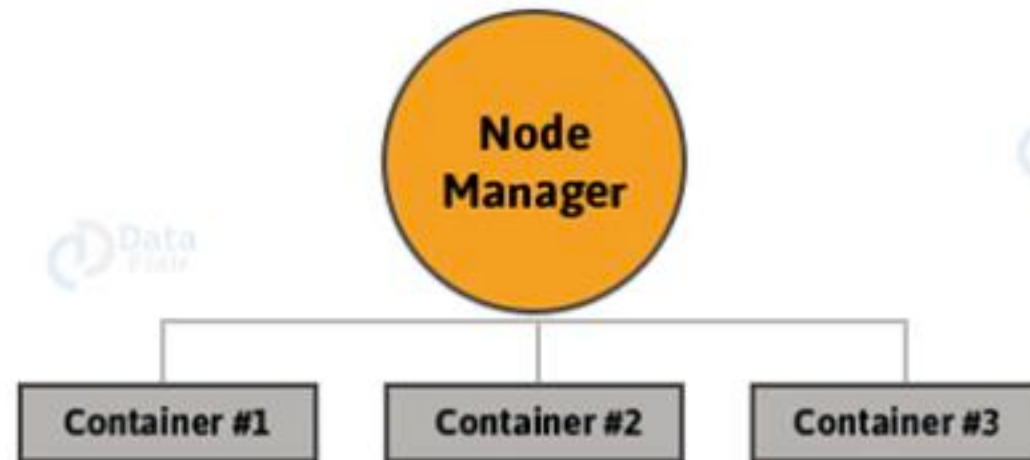




# YARN – Node manager

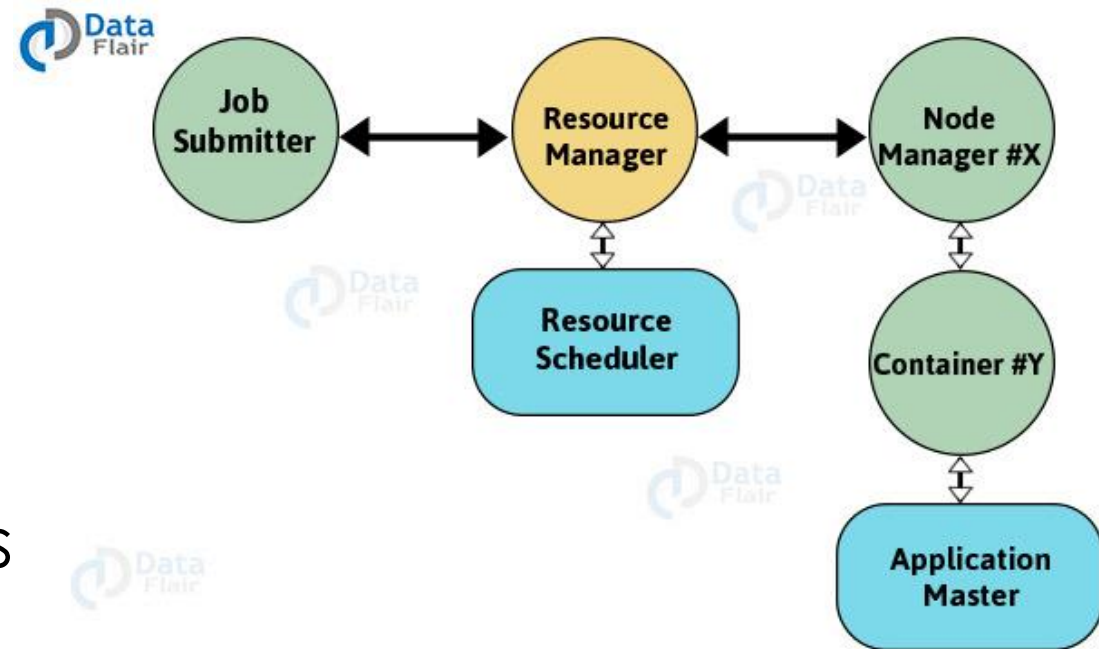
---

- One for each node
- It manages the container, that are fractions of node's computational resources and checks their resources' usage
- Sends the heartbeat signal to the Resource manager



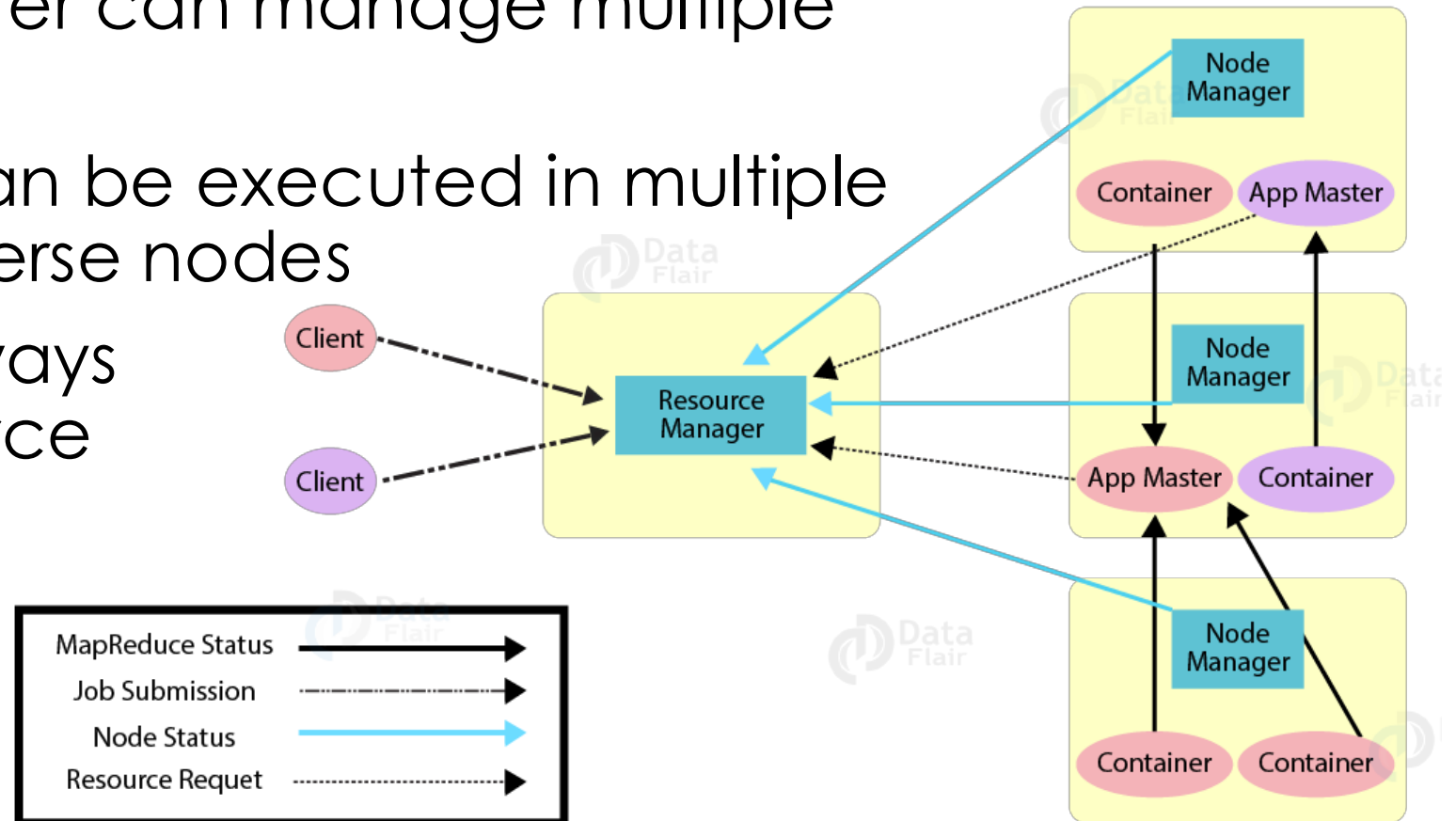
# YARN – Job Submitter

- It submits the job to the Resource manager
- Resource manager and Resource scheduler obtain the first available container for Application Master execution
- Resource Manager contacts the target node for starting the container and execution

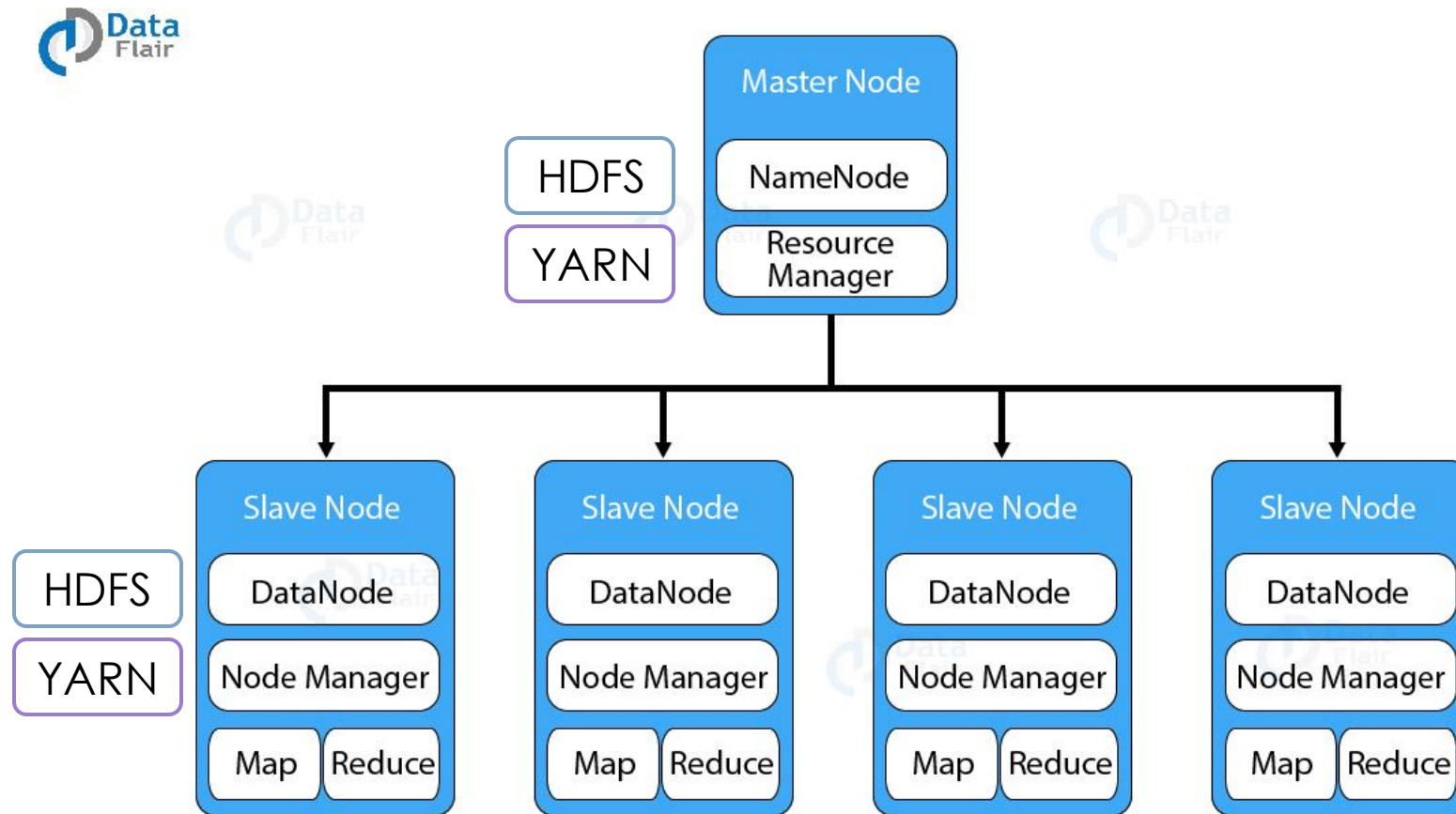


# YARN

- Application Master can manage multiple containers
- The same task can be executed in multiple containers in diverse nodes
- Requests are always made via Resource Manager



# Overall architecture



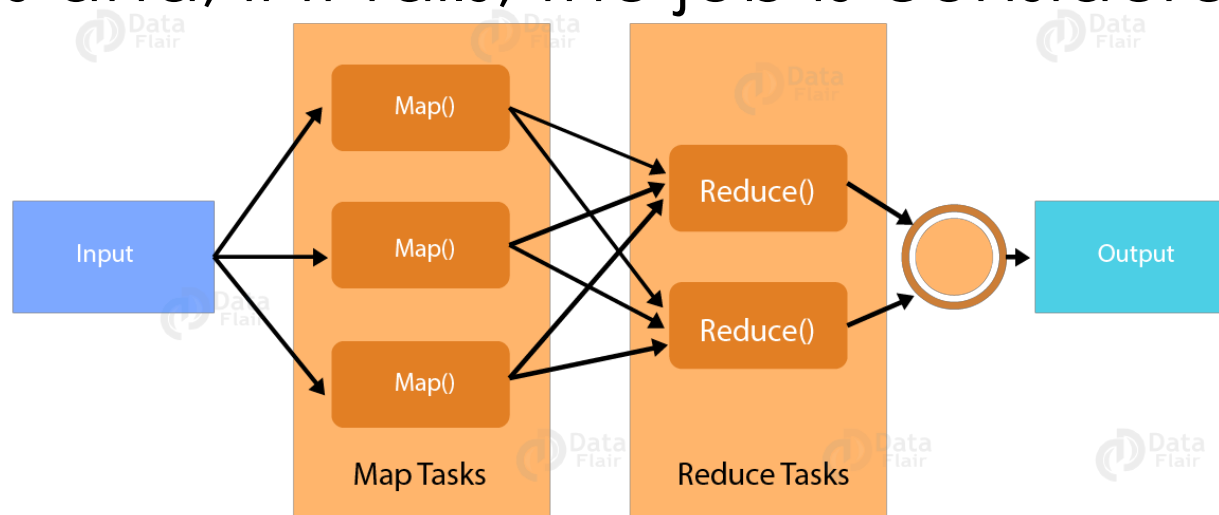
# MapReduce

---

- MapReduce manages the execution of programs on the data present in HDFS and saves the results back to HDFS
- Each program consists of two components
  - Mapper
  - Reducer
- A task is the execution of the mapper or reducer on a partition of the data present on a node in compliance with the principle of data locality

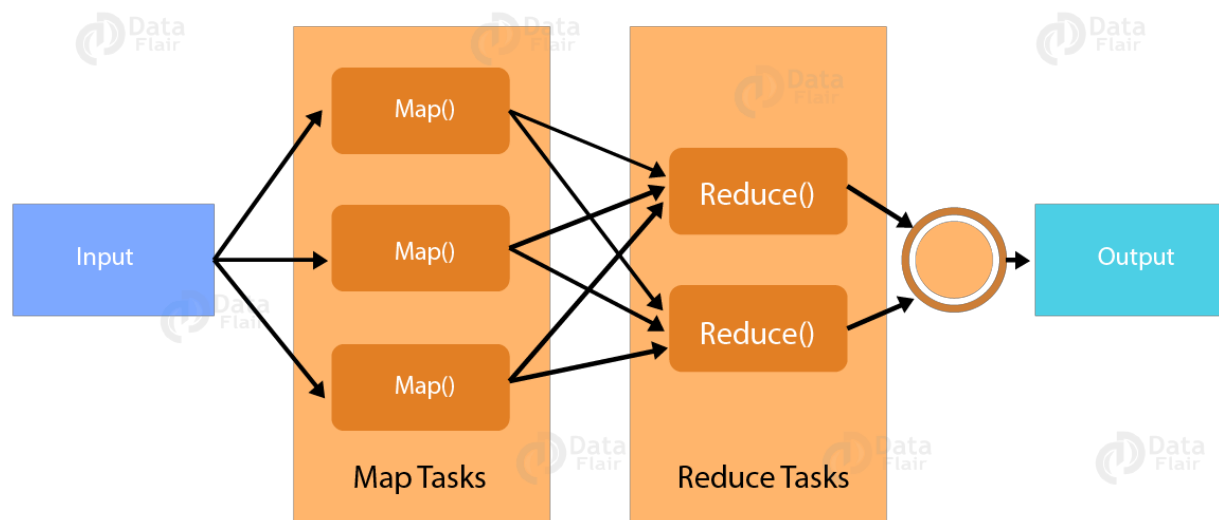
# MapReduce task

- Individual tasks are generated since MapReduce code is migrated to the various slave nodes where the data is located
- The execution of a task is attempted (TaskAttempt) at most 4 times and, if it fails, the job is considered failed



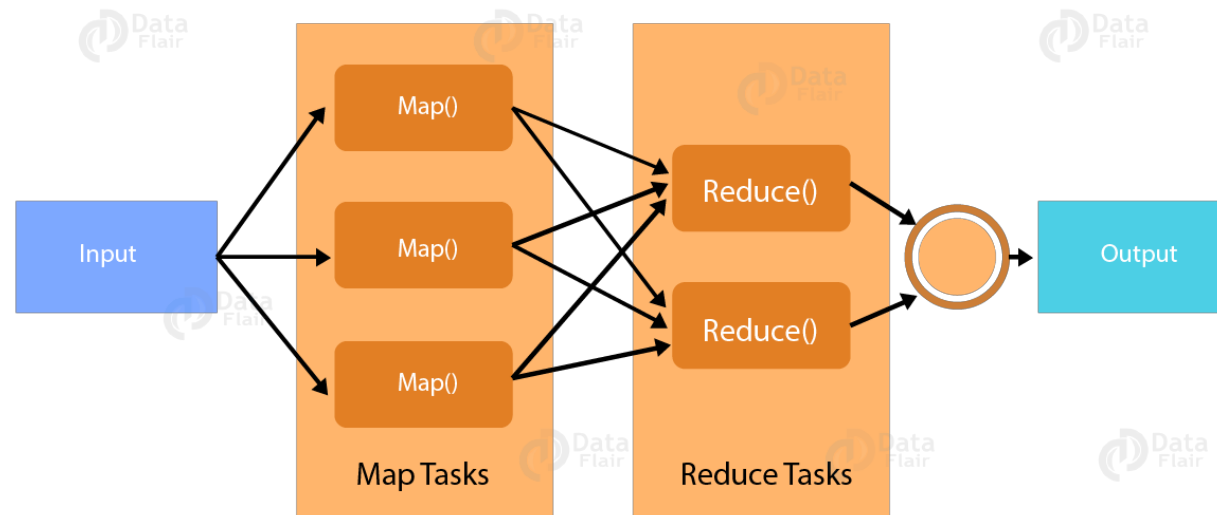
# MapReduce task

- In order to increase performance, speculative execution detects slow tasks and generates backup tasks. When the faster of the two finishes the other is cancelled



# MapReduce task

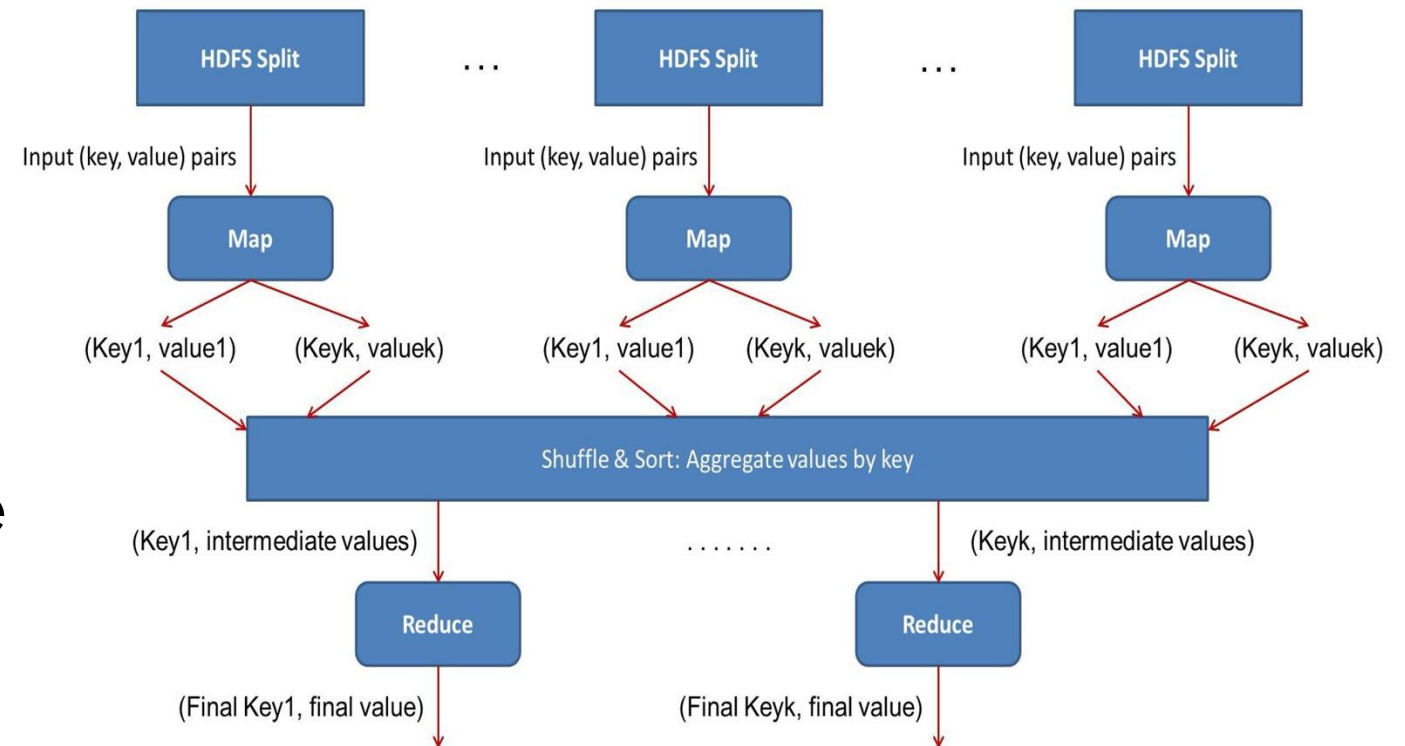
- Map and reduce tasks are typically executed on separate nodes
- The entire process is done by processing key value pairs





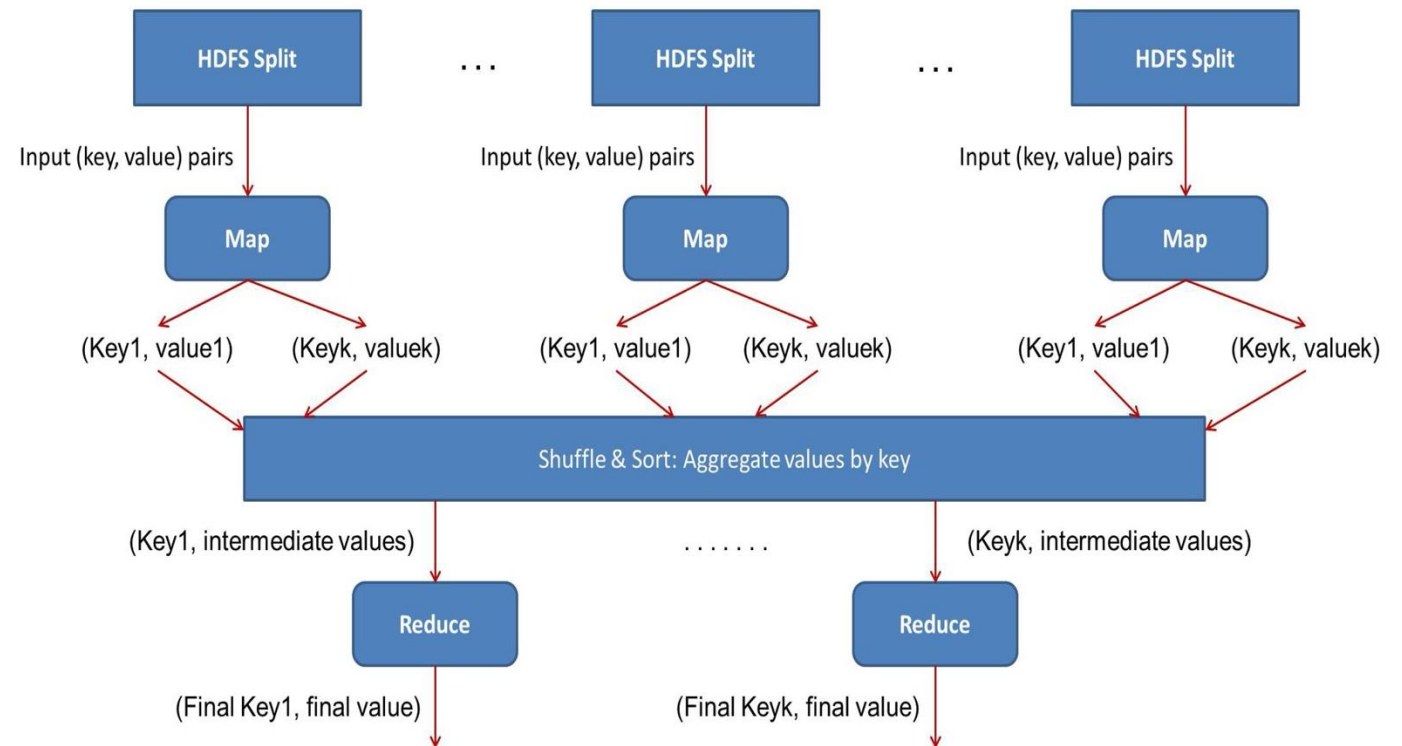
# MapReduce flow

- HDFS splits data, into 128MB blocks
- Each Map task works on all of its input consisting of key-value pairs, that are after transformed into newer key, value pairs, according to the developed programming logic



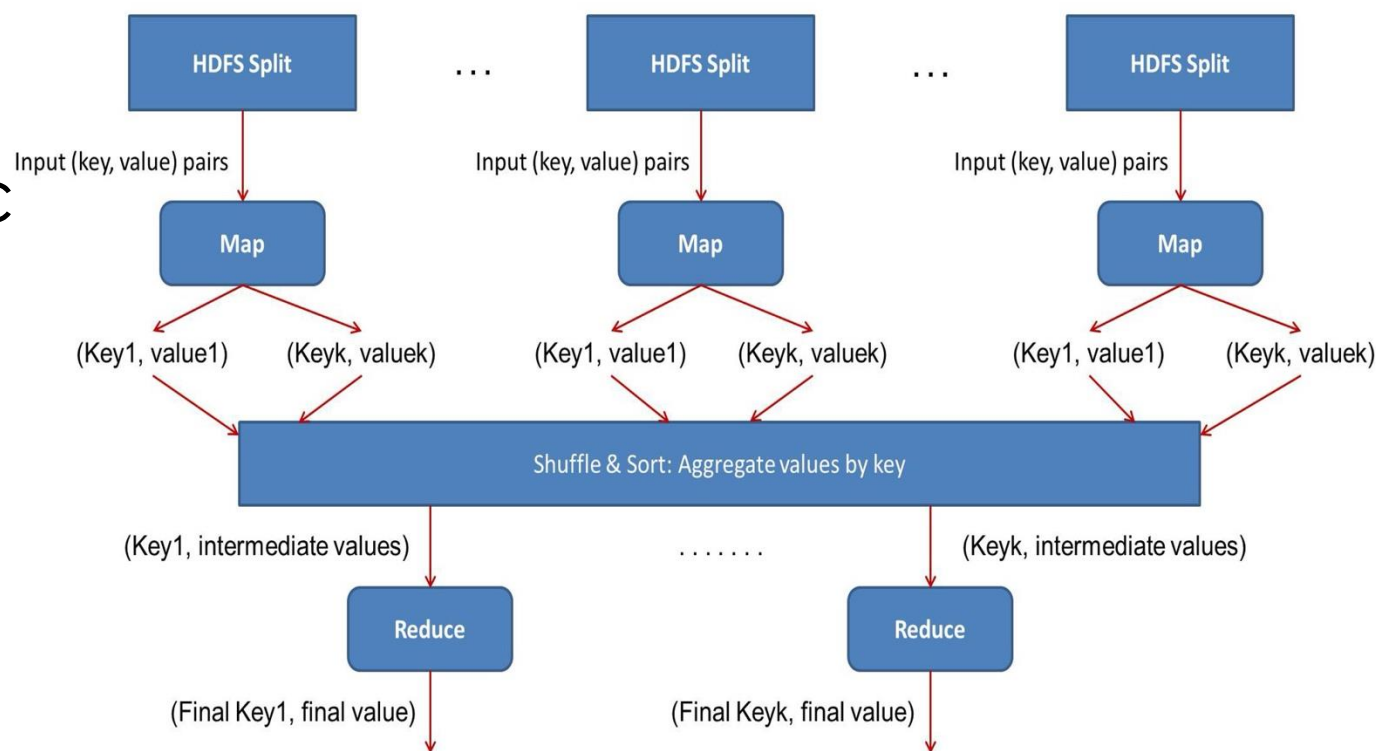
# MapReduce flow

- Outgoing pairs from all Map tasks are processed by a shuffle/sort layer that aggregates data by key
- This phase has the scope to reduce nodes for Reduce task



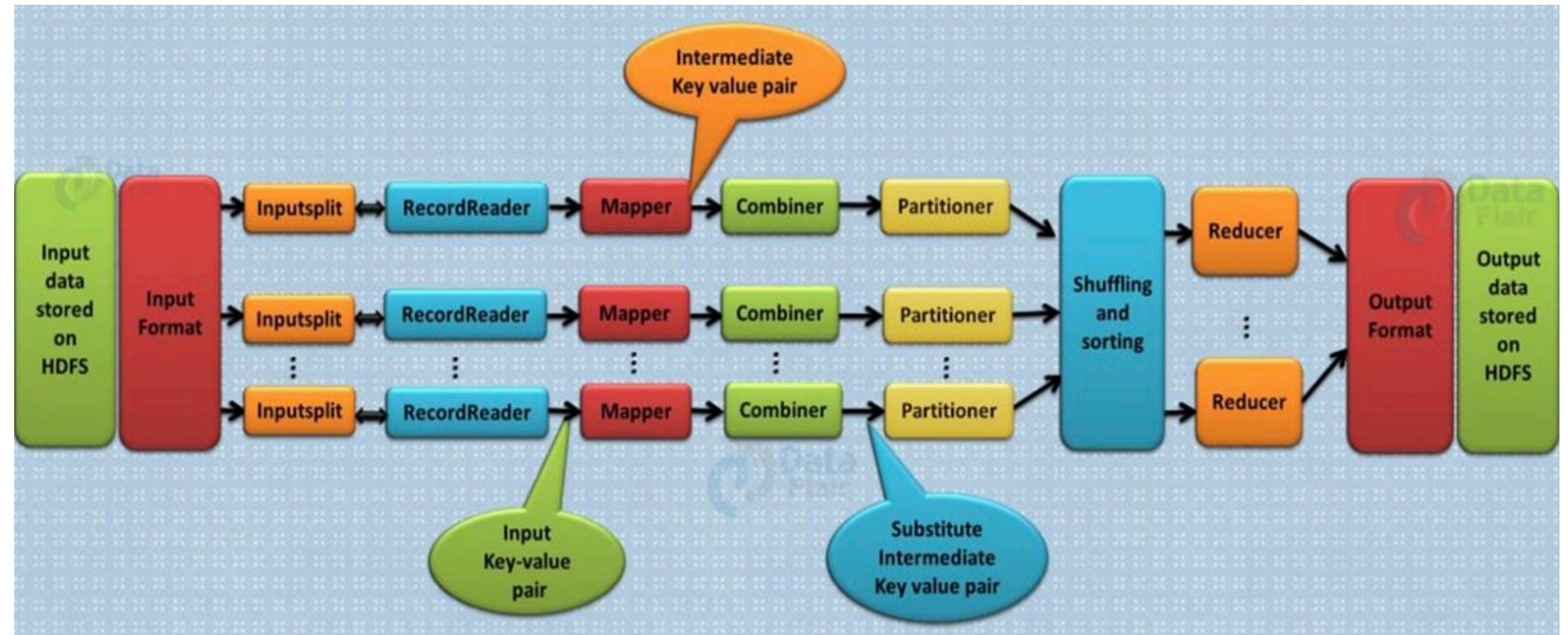
# MapReduce flow

- Each Reduce task generates a new list of pairs according to its programmed logic
- Obtained pairs in the final stage is usually different from the intermediate values
- Result saved back to HDFS



# MapReduce software architecture

- Execution pipeline mapped to a set of Java classes
- A MapReduce program has a fixed structure within which the actual logic is introduced



# MapReduce software architecture

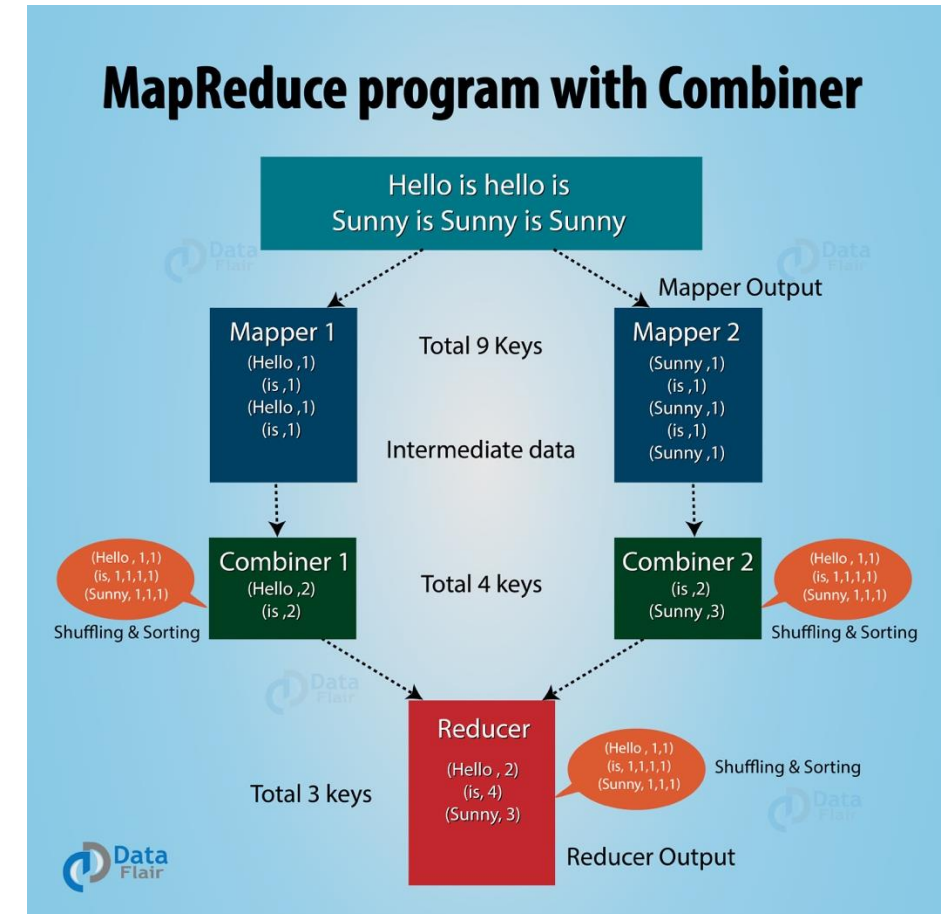
---

- Key-value pairs are the mechanism on which MapReduce processing is based
- They are defined by the user in order to analyze the data, thus they are not intrinsic properties of the data itself
- There are several input and output formats handled by MapReduce that can be used by the user



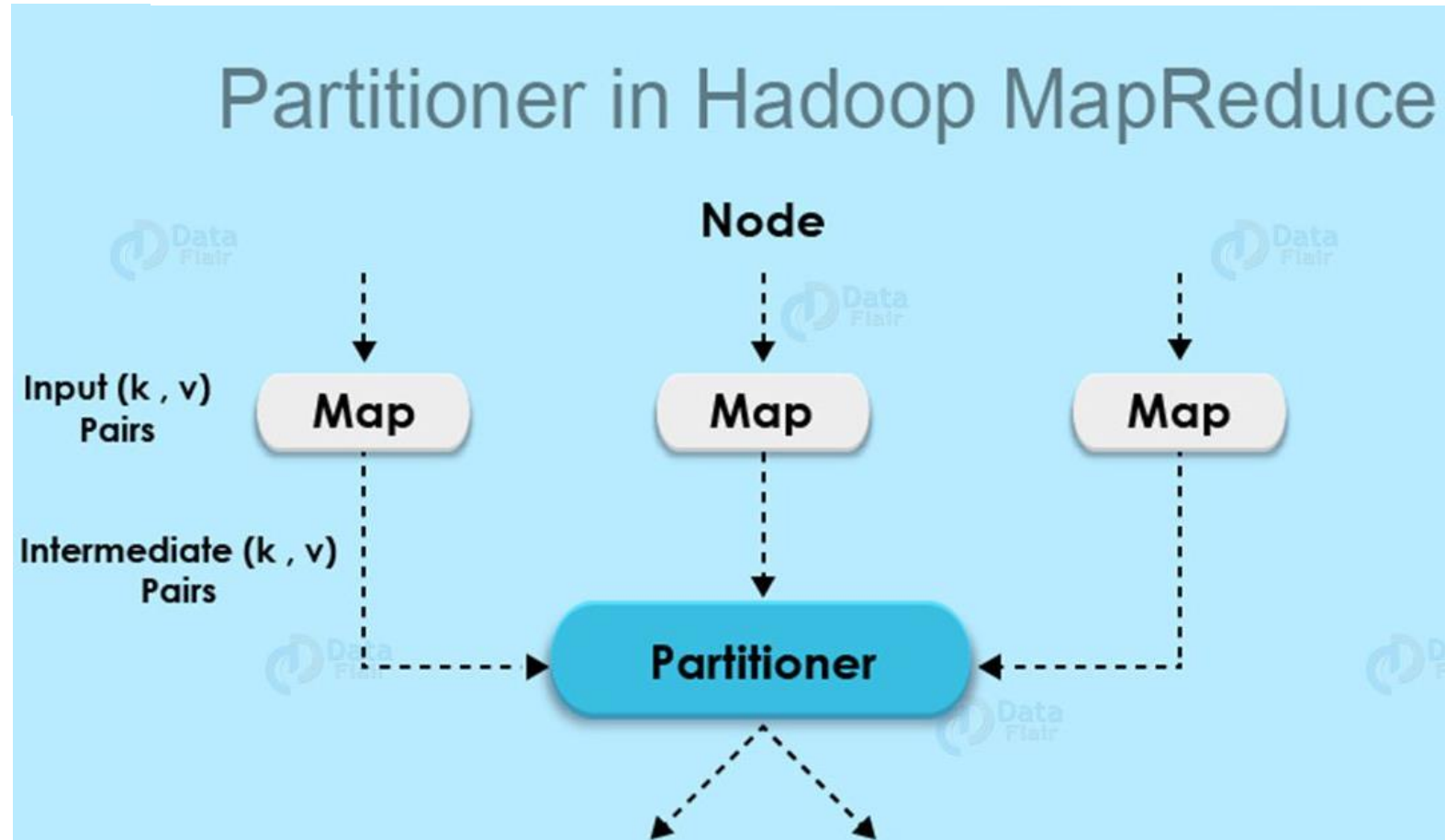
# MapReduce software architecture

- Combiner is a local reducer that shuffles & sorts generated pairs by the mapper
- Decreases network traffic and reducer load
- Combiner does not avoid the shuffling & sorting phase of the reducer
- Can lead to overuse of the mapper's local disk



# MapReduce software architecture

- The Partitioner creates partitions of key-value pairs to be sent to Reduce tasks using a hashing technique on the key, to group pairs with the same key into the same partition



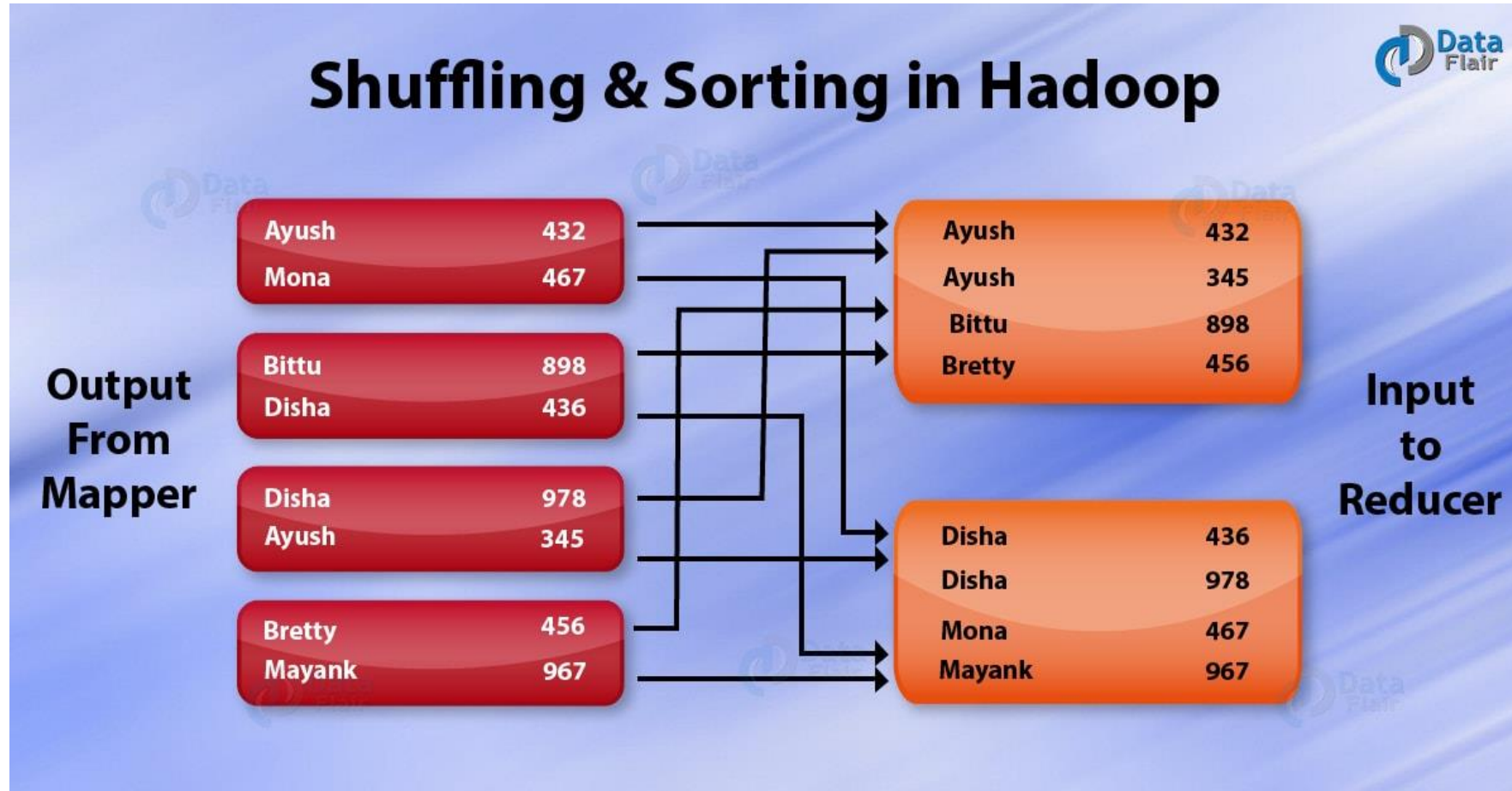
# MapReduce software architecture

---

- Shuffling and sorting operations are performed by the framework
- Shuffling redistributes the values generated by mappers to pass them to reducer
- Sorting takes place by key together with shuffling and before the reducers are started
- Each reducer does not need to do sorting
- Sorting is used by the framework to determine the number of reducers



# MapReduce software architecture



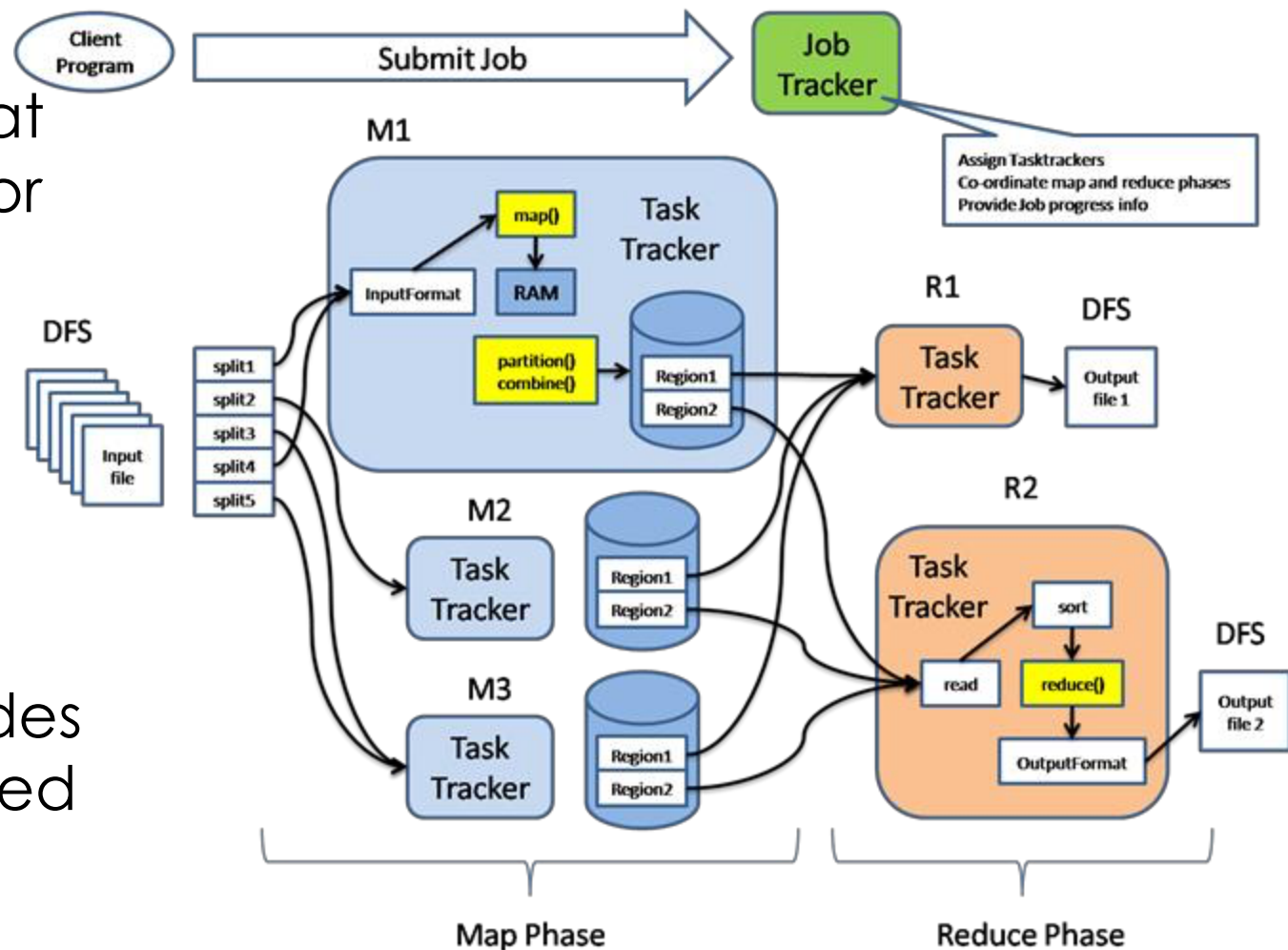
# MapReduce Engine

## Job

- This is the actual work that needs to be performed or processed

## Task

- This is a part of the actual work that needs to be performed or processed.
- A MapReduce job includes many small tasks that need to be executed.



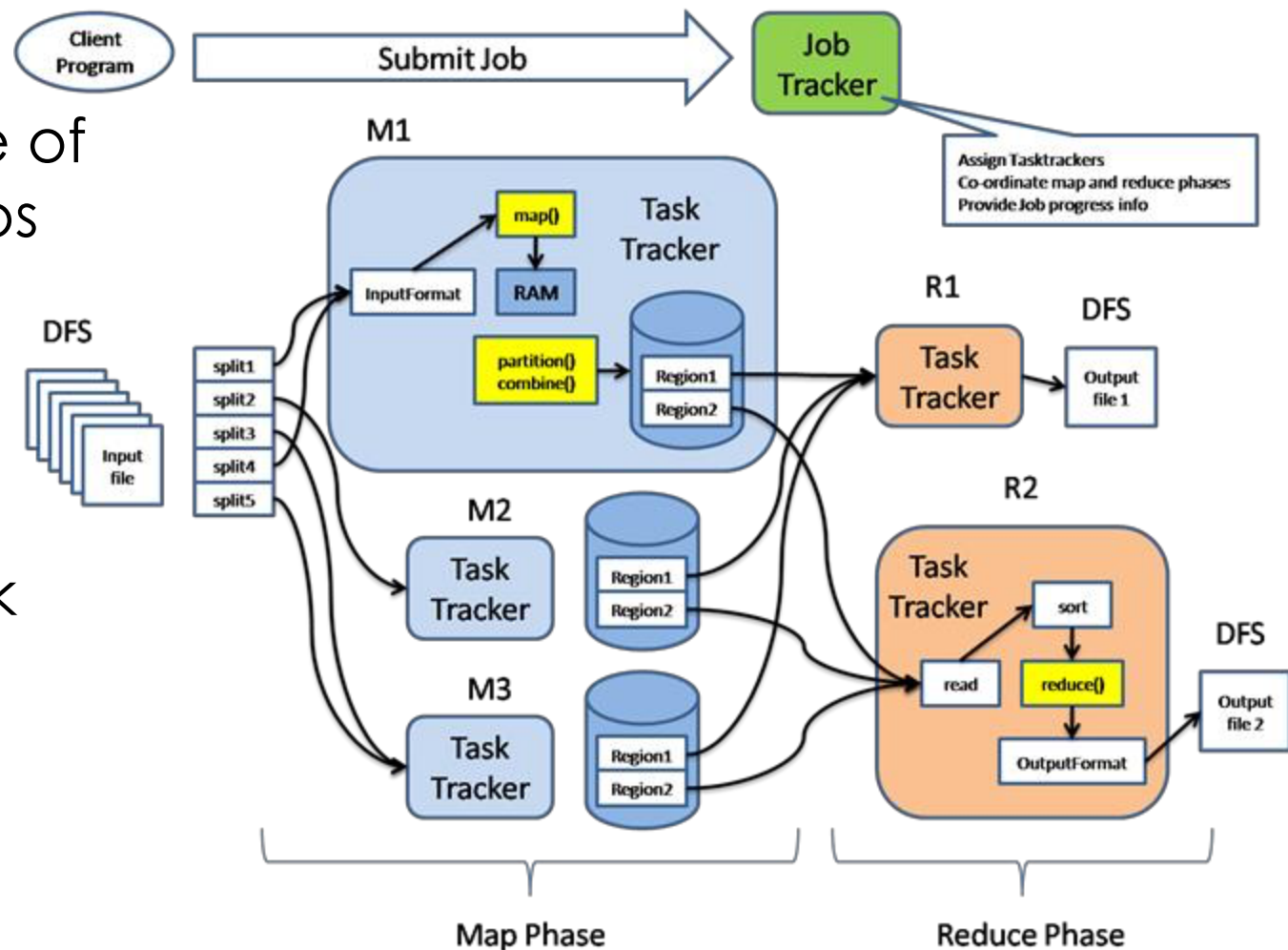
# MapReduce Engine

## JobTracker

- This tracker plays the role of job scheduling and keeps track of all the jobs assigned to the TaskTrackers.

## TaskTracker

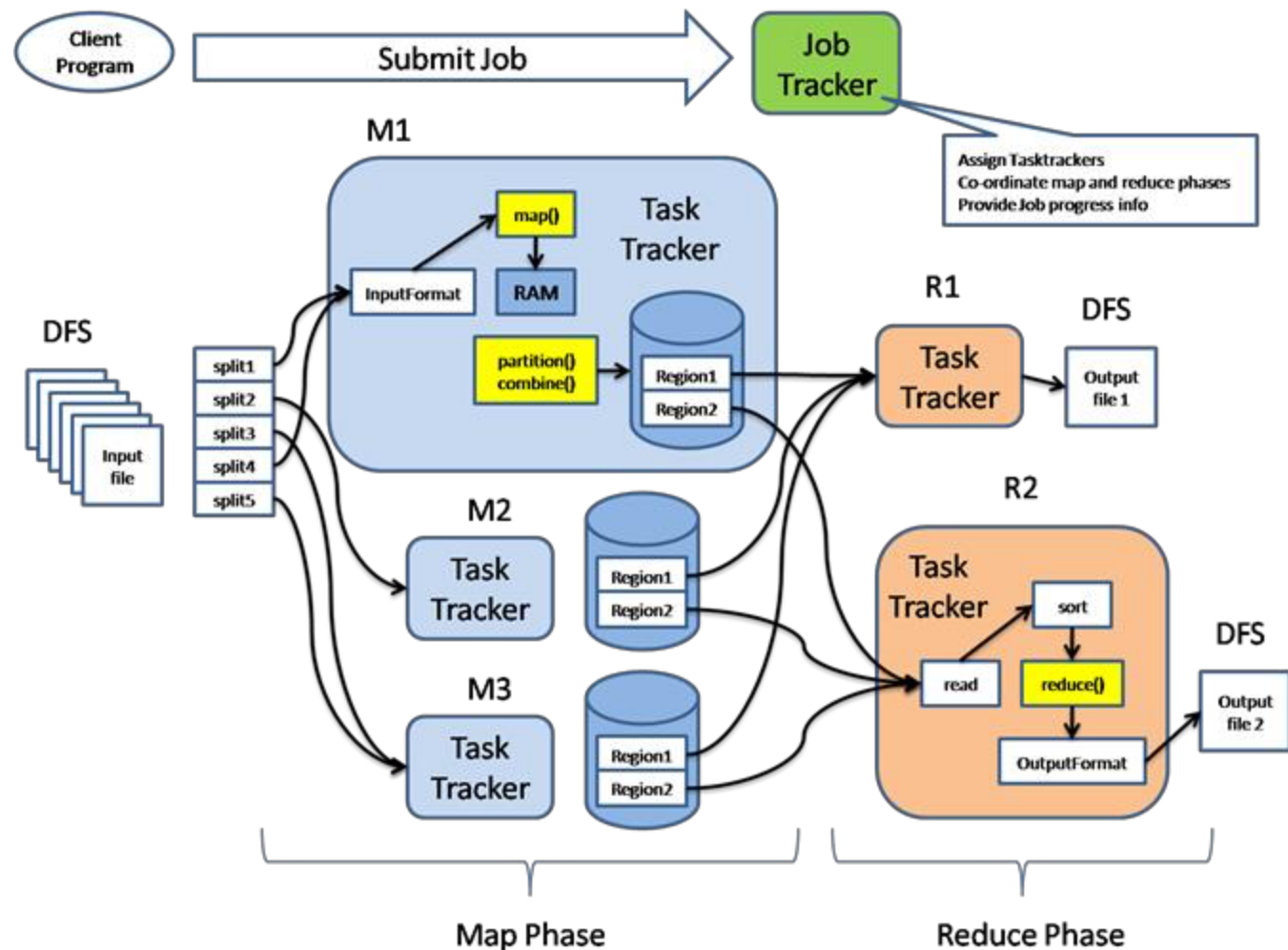
- This tracker acts as a task tracker and reports the status of tasks to the JobTracker.



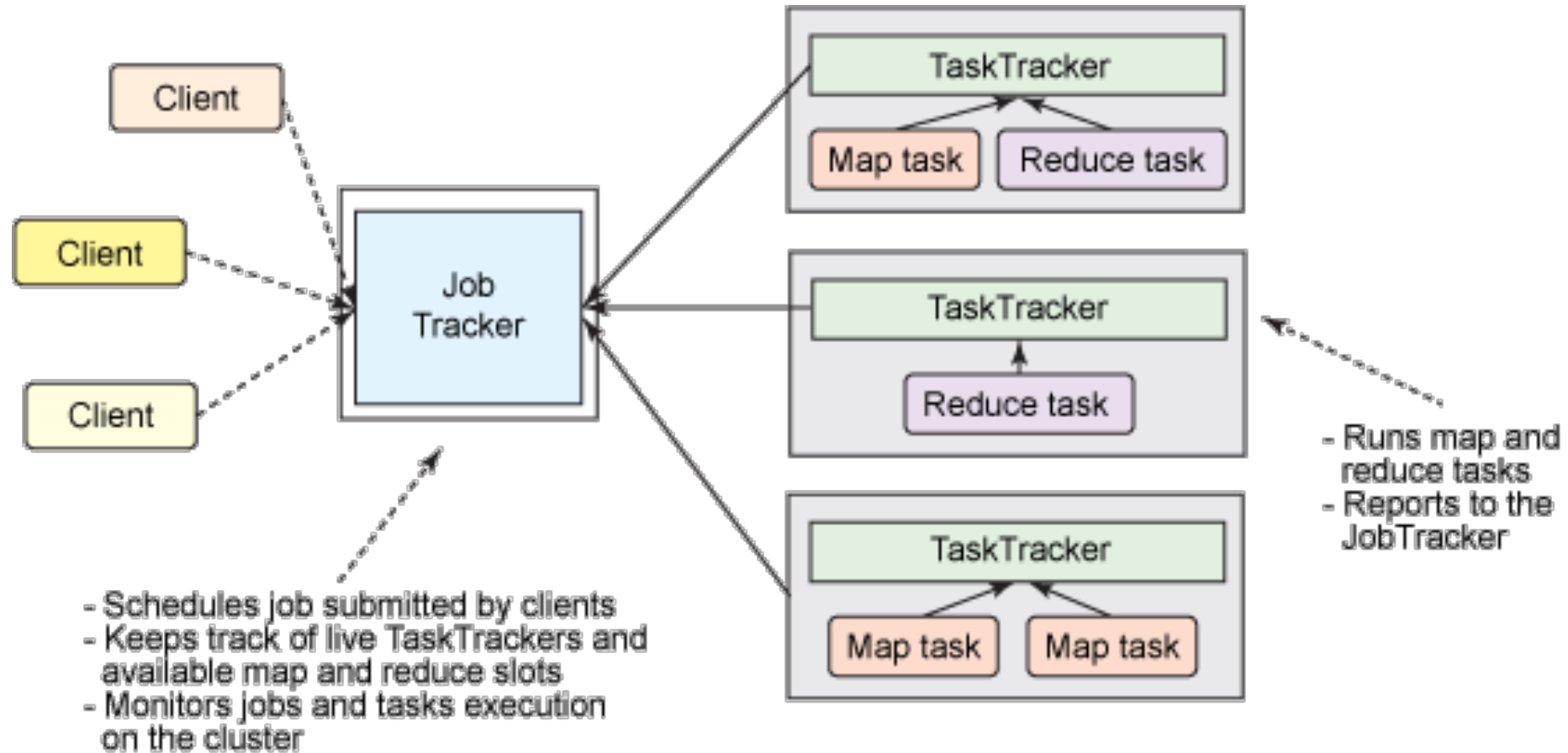
# MapReduce Engine

## Client

- This is a program or application programming interface (API) that sends jobs to MapReduce.
- MapReduce can accept jobs from different clients.

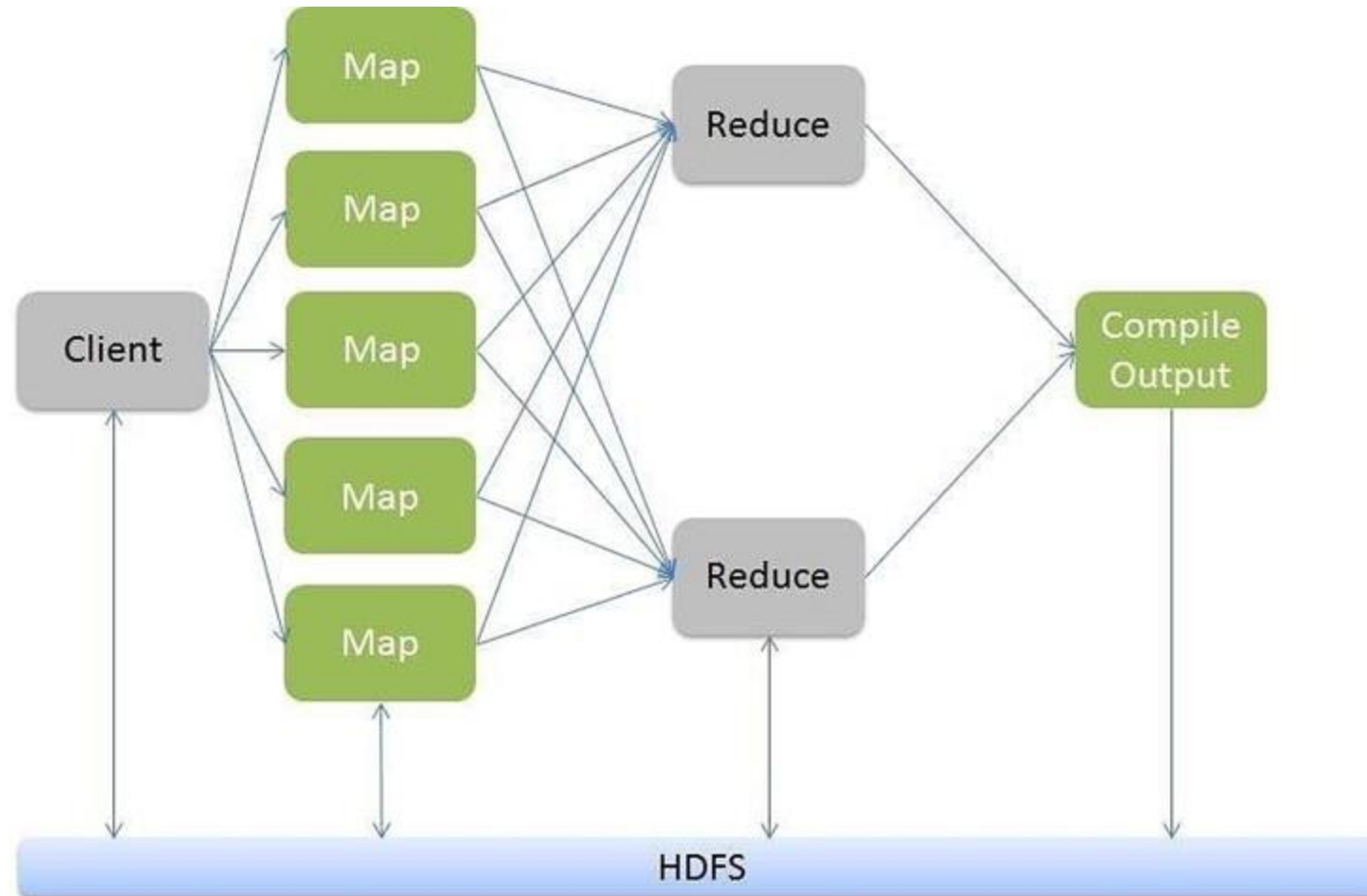


# JobTracker and TaskTracker

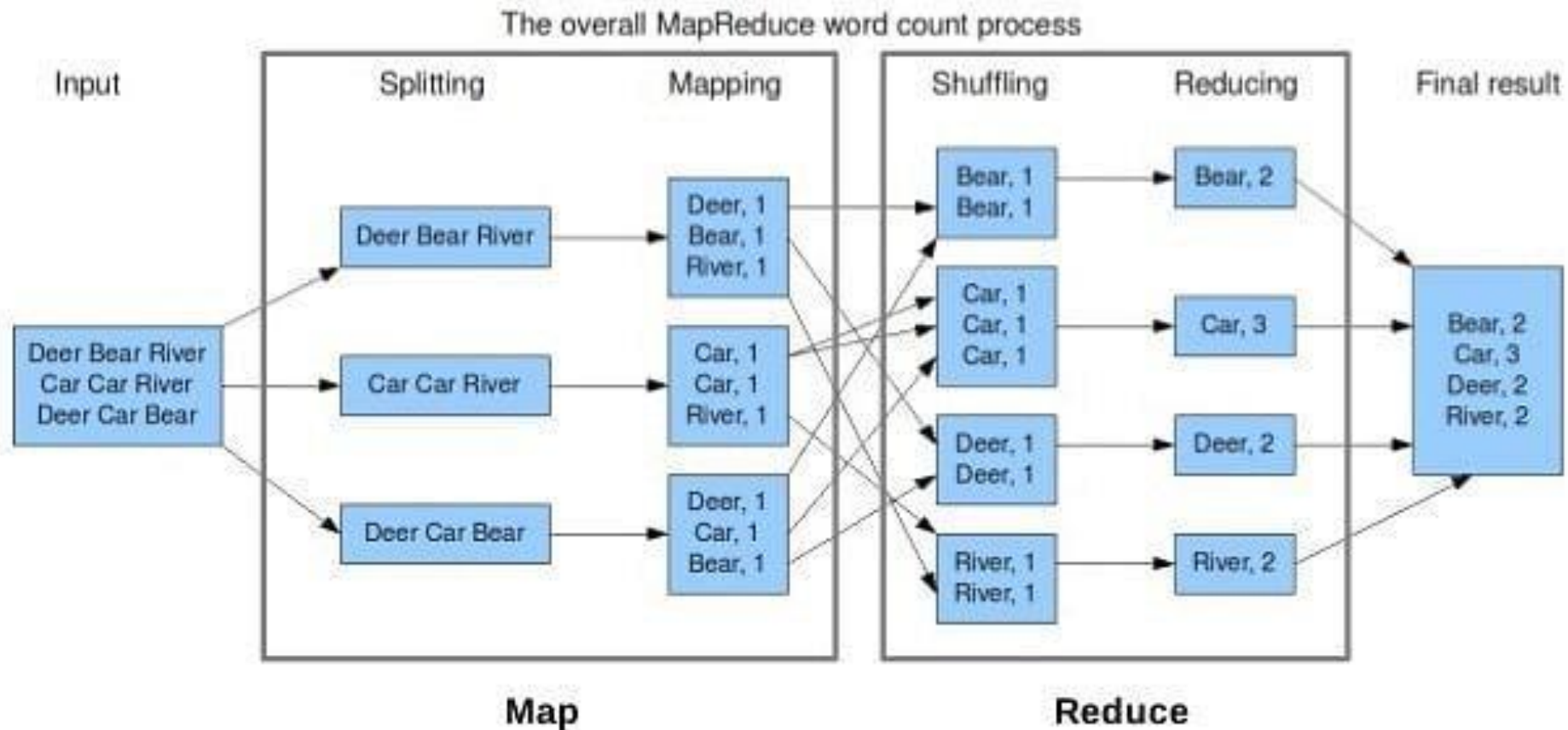




# MapReduce example



# MapReduce – Word count



# MapReduce pros

---

- ✓ Scalability, since the same code can be executed over a cluster with single or thousands of nodes
- ✓ Shared nothing-paradigm that prevents synchronization and sharing memory issues
- ✓ Fault tolerance in case of failures
- ✓ HDFS filesystem



# MapReduce cons

---

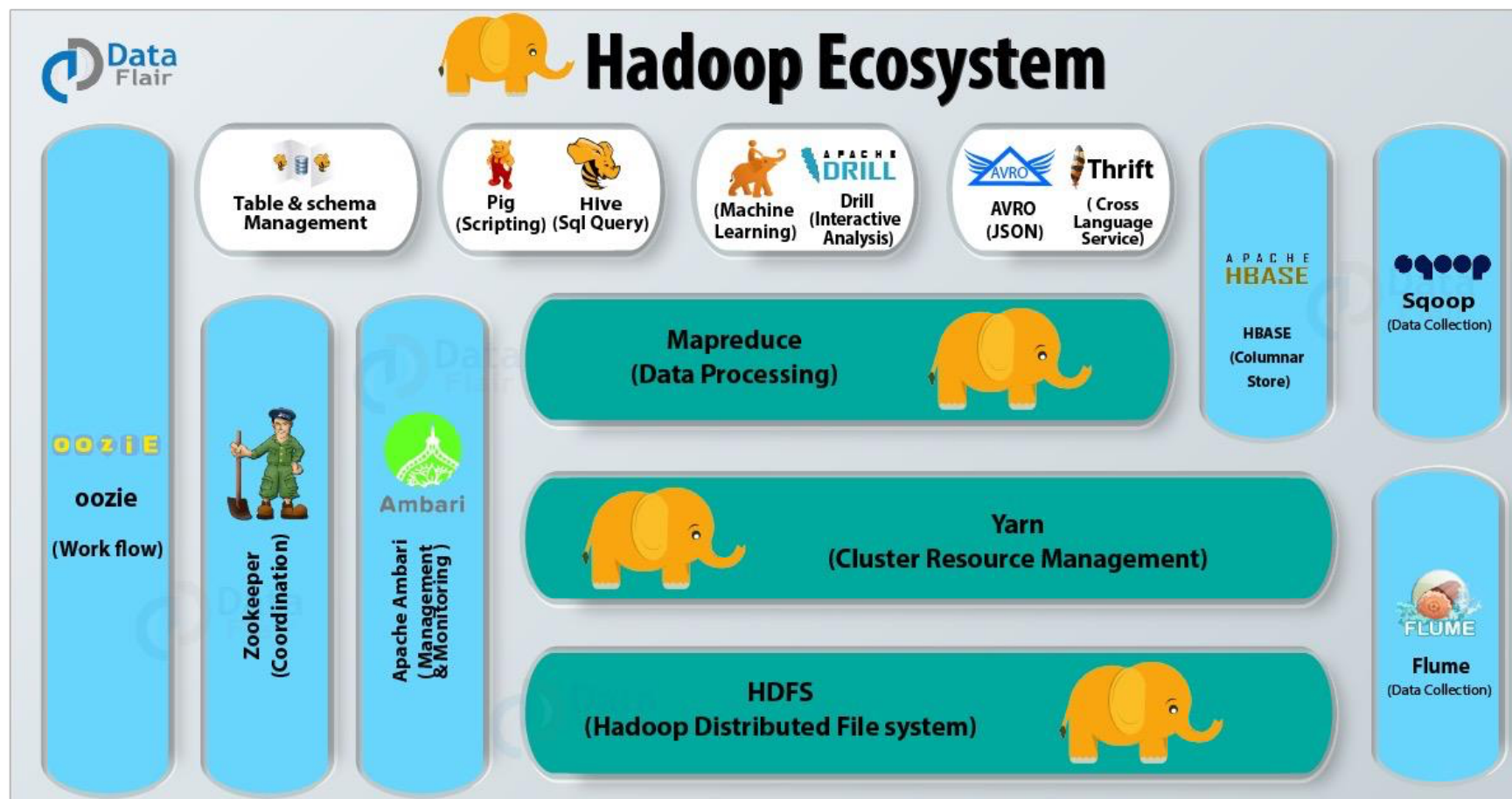
- ✗ Too much I/O operations
- ✗ Strict execution flow (map, shuffle and sort, reduce)
- ✗ Initial overhead at every MapReduce job
- ✗ It is not optimal for an iterative flow of operations (as ML applications)
- ✗ Manual management of low-level details
- ✗ Simple operations became complex

# MapReduce focus on

---

- Principal memory at cluster level is often not used:
  - Data to be analyzed can be kept in RAM memory at cluster node level, thus the overall memory cost decreases
- Some I/O operations are redundant, especially in case of iterative algorithms
  - Intermediate results are sometimes useless and saving them is a waste of time and resources
- High-level API are needed to make easy most common operations

# Hadoop ecosystem



# Hadoop usage

---

- Advertising
  - Analysis of user behavior to generate targeted recommendations
- Searches
  - Relationships between groups of documents
- Security
  - Search for uncommon patterns
- Processing large non-real-time datasets:
  - The NY Times dynamically generated PDFs of articles from 1851 to 1922
  - Wanted to pre-generate and statically publish articles to improve performance
  - Using Hadoop + MapReduce converted 4 TB of TIFF into 11 million PDF articles in 24 hours